



9.1 迁移工具

数据迁移概述

- 数据迁移是指将数据从一个数据库迁移至另一个数据库，可以是同构数据库之间的迁移，也可以是异构数据库之间的迁移。
- 按照数据的流向来分类，数据迁移分为数据导出和数据导入两种操作，通常会存在一种中间态文件，例如SQL文件、CSV文件等，中间态文件可保存在磁盘上，需要时再导入目标数据库中，可实现数据导出与导入的解耦。
- 按照迁移是否停止应用来分类，数据迁移分为冷迁移和热迁移。冷迁移即为停服迁移，其操作简单，但需停止业务，不适用于实时在线业务；热迁移对用户业务影响较小，业务应用无感知，但迁移过程相对比较复杂。
- 同构数据库迁移相对比较简单，可借助备份恢复技术（逻辑备份恢复和物理备份恢复）实现迁移。
- 异构数据库迁移相对比较复杂，需综合考虑异构数据库的兼容性、选型、应用改造成本、迁移成本、性能等众多因素。
- 本次课程主要介绍异构数据库（MySQL->openGauss）之间的热迁移。

数据迁移五步曲

能力现状

迁移评估

应用适配

数据迁移

试运行

生产割接

兼容评估工具

- 工具对内核语法做兼容性评估
- 数据库调研表，调研部署架构、典型业务、数据容量、功能、性能。

兼容性

- 兼容常用语法100%，全量语法80%兼容，共计475+个；支持包括无符号整数、enum/set数据类型、时间数据类型、自增列、json函数、replace into等常用语法的兼容。

迁移工具

- gs_mysync工具：表、索引、存储过程等对象全量迁移及数据全量迁移；
- gs_replicate工具：建表、修改表等常见DDL增量迁移；DML增量迁移；
- gs_datacheck工具：基于默克尔树的数据实时校验，支持全量和增量数据校验。

监控诊断

- 故障诊断：日志分析、慢SQL诊断；
- 数据库监控：监控数据库各项指标共100+视图。

反向迁移、监控工具

- 反向迁移工具，支持openGauss到MySQL迁移，满足迁移后逃生要求；
- 监控工具，监测数据库各项指标是否合理。

规划增强

迁移评估

- 工具化实现对MySQL容量、部署架构、负载的评估，生成评估报告。

兼容性

- 完善兼容性，做到应用近似零修改迁移；
- 支持对MySQL常用协议100%兼容，JDBC无缝替换。

迁移工具

- 支持增量迁移断点续传能力，提升迁移可靠性；
- 增强工具易用性和监控能力，支持一体化工具平台DataKit。

流量镜像

- 故障诊断：openGauss作为影子库，MySQL流量1:1分摊到openGauss。

反向迁移、监控工具

- 反向迁移支持断点续传和全量迁移，提升迁移可靠性。

迁移评估（1）

- 结合数据库源库画像，给出目标数据库选型和部署建议；评估目标数据库兼容性，确定应用改造工作量和迁移时长。
- 源库画像用于全方位了解源库，为后续选型评估、数据迁移提供依据和指导。
 - **系统级画像**：收集系统级信息，例如硬件（CPU、MEM、NET、DISK）、操作系统（内核、参数、安全策略等）、性能（系统高峰期24小时负载）等。
 - **实例级画像**：收集实例级信息，例如架构（单机/集群（主备、主从、跨AZ部署、异地部署）、版本等）、参数（数据库参数等）、数据规模（数据库数量及大小、表、索引等空间使用）、运行态信息（如数据库实例的并发量、QPS、TPS、会话、事务、慢SQL、等待事件等）、资源占用（系统CPU、内存、网络IO、磁盘IO等）。
 - **对象级画像**：收集对象级信息，例如结构信息（表、分区、分片、索引、视图、序列、存储过程、自定义函数等）、统计信息、访问特征（读写比、频率等）、预警类（如大表、复杂结构、DBLink、外部表等）、特殊类型（blob、数组、自定义类型）。
 - **语句级画像**：SQL文本（全量）、执行特征（次数、响应时间等）、执行计划、热点（访问频率对象、频繁执行的SQL/存储过程等）。
 - **应用级画像**：计算文本（如存储过程、触发器、函数等）、执行特征（次数、响应时间等）。

迁移评估（2）

- 应用画像收集应用架构、应用于DB的关系、应用访问特征等，为应用改造做好铺垫。
- 根据源库画像，确定目标数据库的选型及部署建议，包括硬件资源、部署方式、运行参数等。
- 评估目标库兼容性，包括对象兼容性、SQL兼容性，给出应用改造建议，并评估改造工作量。
- 结合源库画像、目标库兼容性，评估迁移时长及迁移成本。

全量迁移gs_mysync – 原理

- 全量迁移将MySQL端已有数据和对象迁移至openGauss端。当前openGauss社区官网提供全量迁移工具gs_mysync，本质为chameleon工具，其由python语言开发。
- 全量迁移支持的数据对象：表、表数据、函数、存储过程、触发器、视图。
- 全量迁移的性能：sysbench测试模型，10张无主键表，单表数据量500w，20个读写并发线程迁移，整体性能可达300M/s。
- 全量迁移实现原理：
 - 采用多进程读写分离实现，生产者从MySQL侧读取数据写入CSV文件，消费者读取CSV文件写入openGauss，多个表并行处理
 - 针对大表，会将其分成多个CSV文件，默认一个CSV文件2M
 - 迁移顺序：表结构->表数据->表索引

全量迁移gs_mysync – 实现逻辑

- 记录全量迁移开始快照点
- 创建目标schema及表结构，不包含索引
- 创建多个读写进程，主进程针对每个表创建一个读任务，加入读任务队列
- 读写进程并行执行，读进程记录每个表的快照点，读取表数据存入多个csv文件；写进程同时copy csv文件至目标数据库
- 数据迁移完成后创建索引
- 所有任务完成后关闭读写进程

全量迁移gs_mysync – 使用步骤（1）

- 创建配置文件，并自定义配置

```
chameleon set_configuration_files
```

- 初始化复制，创建复制过程的辅助schema和表

```
chameleon create_replica_schema --config default  
chameleon add_source --config default --source mysql
```

- 复制基础表及数据

```
chameleon init_replica --config default --source mysql
```


全量迁移gs_mysync – 使用步骤（2）

- 复制函数、存储过程、触发器、视图

```
chameleon start_func_replica --config default --source mysql
chameleon start_proc_replica --config default --source mysql
chameleon start_trigger_replica --config default --source mysql
chameleon start_view_replica --config default --source mysql
```

- 在线复制

```
chameleon start_replica --config default --source mysql
```

- 结束复制过程及清理资源

```
chameleon stop_replica --config default --source mysql
chameleon detach_replica --config default --source mysql
chameleon drop_replica_schema --config default
```

增量迁移gs_replicate – 原理（1）

- 增量迁移将MySQL端产生的增量数据（IUD操作及DDL）迁移至openGauss端。当前openGauss社区官网提供增量迁移工具gs_replicate，其由Java语言开发。
- 增量迁移实现原理：基于debezium+kafka实现
 - debezium mysql connector的source端，监控mysql数据库的binlog日志，并将数据（DDL和DML操作）以AVRO格式写入到kafka；
 - debezium mysql connector的sink端，从kafka读取AVRO格式数据（DDL和DML操作），并组装为事务，在openGauss端按照事务粒度并行回放，从而完成数据（DDL和DML操作）从mysql在线迁移至openGauss端；
 - 由于该方案严格保证事务的顺序性，因此将DDL和DML路由在kafka的一个topic下，且该topic的分区数只能为1（参数num.partitions=1），从而保证source端推送到kafka，和sink端从kafka拉取数据都是严格保序的。

增量迁移gs_replicate – 原理（2）

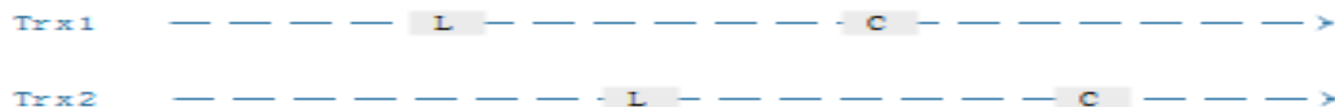
- 增量迁移的性能：利用sysbench对MySQL进行压测，2路鲲鹏920 CPU、openEuler操作系统下，针对混合IUD场景，10张表50个线程（insert-30线程，update-10线程，delete-10线程），在线迁移性能可达3w tps。
- 增量迁移并行机制：source端日志解析和sink端日志回放双并行。
- Source端并行原理：
 - 开源组件mysql-binlog-connector-java用于解析mysql端的binlog日志为event，原始实现方案为串行解析，通过对源代码进行修改，支持并行解析event事件，以提高debezium mysql connector作为source端的性能。
- Sink端并行原理：
 - 事务粒度并行回放：基于MySQL的主备并行复制机制实现并行回放。
 - 表级粒度并行回放：基于表粒度并行实现并行回放。

增量迁移gs_replicate - 事务粒度并行回放 (1)

- 事务粒度并行回放：基于MySQL的主备并行复制机制实现并行回放。
- 主备复制延迟是一大痛点问题，提高并行度是降低延迟的关键因素，伴随着MySQL的演进，并行度一直在逐步提升。
- 事务粒度并行回放：基于MySQL的主备并行复制机制实现并行回放。
- 如果两个事务能在同一段时间内获取各自的锁，说明能并行执行。
- lock interval表示一个事务持有锁的时间间隔：
 - 当最后一把锁获取，lock interval开始；
 - 当存储引擎提交，第一把锁释放，lock interval结束。

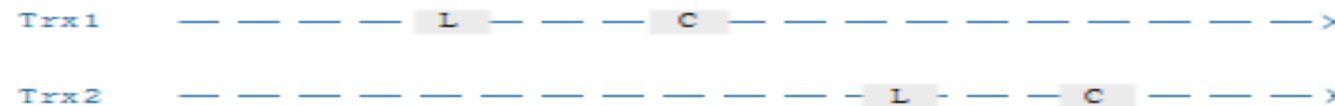
增量迁移gs_replicate - 事务粒度并行回放 (2)

- 可以并行的事务

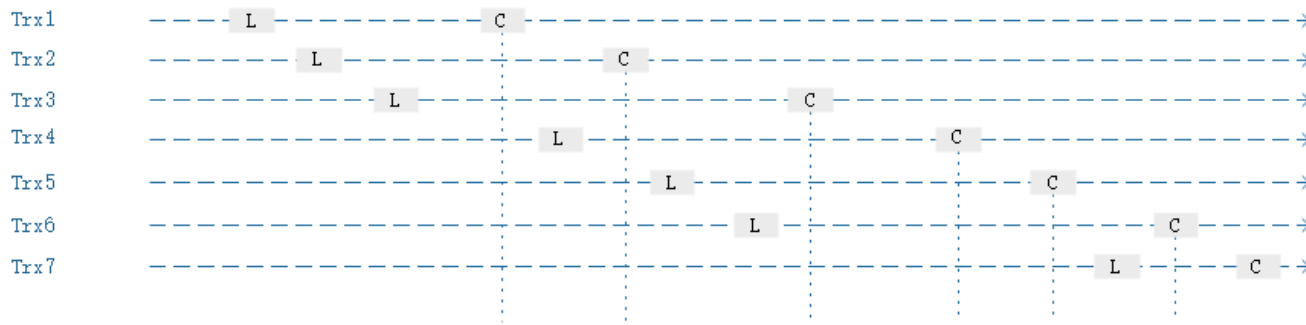


- 当Trx1、Trx2的lock interval有重叠，则可以并行执行

- 不可以并行的事务



- Trx1结束的lock interval要早于Trx2开始的，两者无重叠，不可以并行执行



- Trx3、Trx4、Trx5、Trx6四个事务可以并发执行
- 当Trx3、Trx4、Trx5执行完成之后，Trx6和Trx7可以并发执行
- Trx5和Trx7不能并发执行

增量迁移gs_replicate - 事务粒度并行回放 (3)

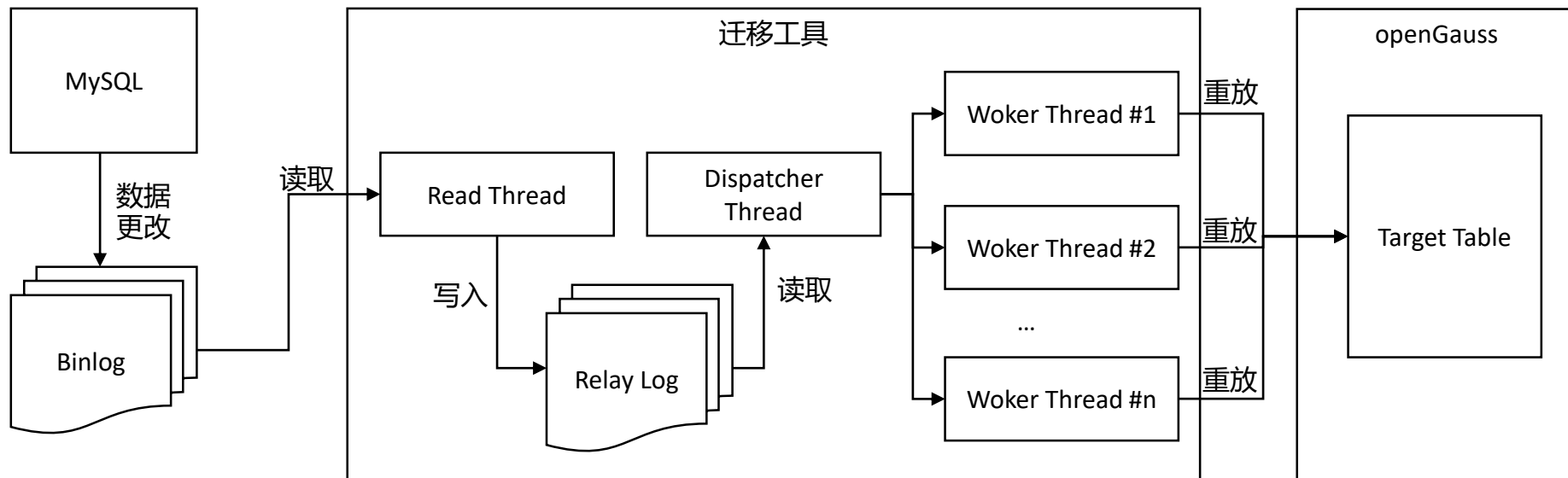
- 事务粒度并行回放：基于MySQL的主备并行复制机制实现并行回放
 - last_committed和sequence_number
 - 在Binlog中会记录每个事务的lock interval开始点 (L) 和结束点 (C) , 分别取名为last_committed和sequence_number

```
mysqlbinlog --no-defaults /home/mysql/data/mysql-bin.000011 --database=mysql_database --base64-output=decode-rows -v | grep last_committed
```

```
[root@localhost logs]# mysqlbinlog --no-defaults /home/mysql/data/mysql-bin.000011 --database=mysql_database --base64-output=decode-rows -v | grep last_committed
WARNING: The option --database has been used. It may filter parts of transactions, but will include the GTIDs in any case. If you want to exclude or include transactions, you should use the options --exclude-gtid
s or --include-gtids, respectively, instead.
#220113 10:17:29 server id 100  end_log_pos 275 CRC32 0x7488ffb5      GTID      last_committed=0      sequence_number=1      rbr_only=yes      original_committed_timestamp=1642040249605077      immediate_commit_tim
estamp=1642040249605077 transaction_length=543
#220113 10:21:04 server id 100  end_log_pos 818 CRC32 0xbf3c35fb      GTID      last_committed=1      sequence_number=2      rbr_only=yes      original_committed_timestamp=1642040464065616      immediate_commit_tim
estamp=1642040464065616 transaction_length=311
#220113 10:21:06 server id 100  end_log_pos 1129 CRC32 0xd8fbc39f      GTID      last_committed=1      sequence_number=3      rbr_only=yes      original_committed_timestamp=1642040466511441      immediate_commit_tim
estamp=1642040466511441 transaction_length=311
#220113 10:22:34 server id 100  end_log_pos 1440 CRC32 0xa8e0ec26      GTID      last_committed=3      sequence_number=4      rbr_only=yes      original_committed_timestamp=1642040554400566      immediate_commit_tim
estamp=1642040554400566 transaction_length=311
#220113 10:22:38 server id 100  end_log_pos 1751 CRC32 0x7c22185c      GTID      last_committed=3      sequence_number=5      rbr_only=yes      original_committed_timestamp=1642040558463906      immediate_commit_tim
estamp=1642040558463906 transaction_length=311
#220113 10:22:42 server id 100  end_log_pos 2062 CRC32 0x81533fb8      GTID      last_committed=3      sequence_number=6      rbr_only=yes      original_committed_timestamp=1642040562161641      immediate_commit_tim
estamp=1642040562161641 transaction_length=311
```

增量迁移gs_replicate - 事务粒度并行回放 (4)

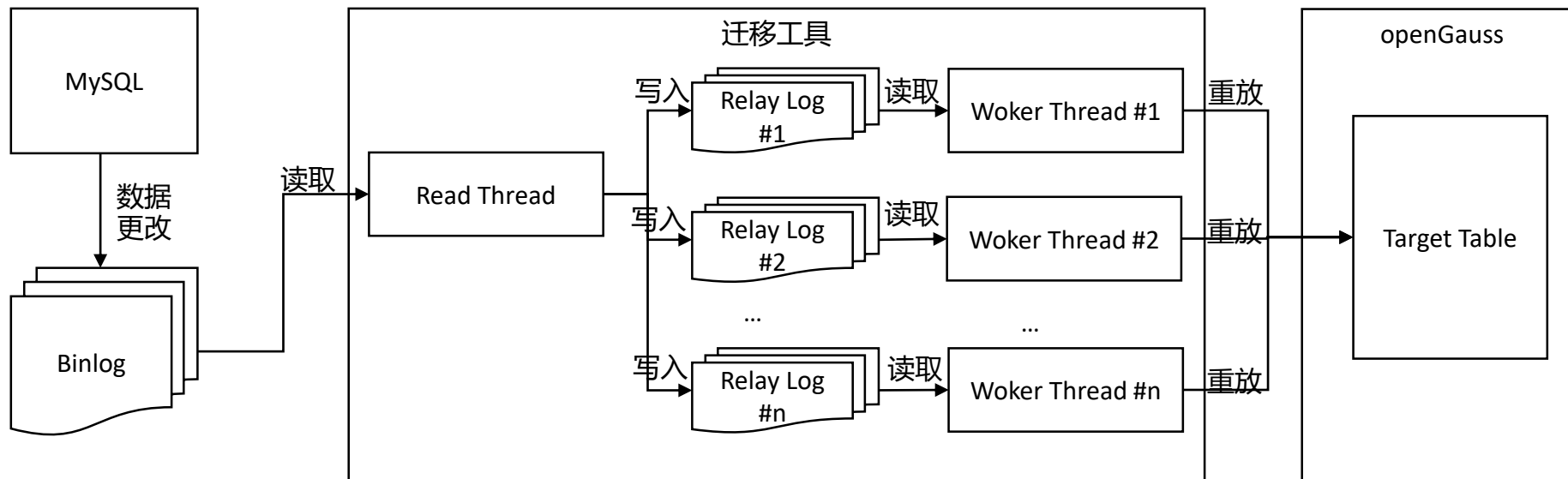
- 事务粒度并行回放：基于MySQL的主备并行复制机制实现并行回放



- 启动一个 Dispatcher 线程负责分发待回放日志。每获取一条日志，需判断它是否能与当前正在回放的事务并行执行，判断规则为：如果所有正在回放的事务的最小sequence_number大于该事务的last_committed，那么该事务就可以并发执行。
- 如果能并行则直接从Woker线程组中寻找一个空闲线程处理；如果不能并行则等待，直到满足条件后，再从Woker线程组中寻找一个空闲线程处理。
- 如果没有空闲Woker线程，需要等待，直到有事务回放完释放Worker线程。

增量迁移gs_replicate – 表级粒度并行回放 (5)

- 表级粒度并行回放：基于表粒度并行实现并行回放



- Read线程在读取binlog后，根据“库+表”进行Hash，根据Hash值写入到多个Relay Log中。
- 启动一组Worker线程，并发从对应的Relay Log文件中获取日志，解析日志构造SQL语句并最终在openGauss端回放。
- 基于“库+表”实现并行，不能保证事务之间的顺序，同时，对于Worker线程负载不均的情况下，会降低整个回放的性能。

校验工具gs_datacheck – 原理（1）

- 校验工具实现原理
 - 校验工具 gs_datacheck 采用JDBC方式抽取数据。
 - 对表原始数据进行Hash计算，并将中间态数据暂存到kafka Topic中。
 - 使用源端和目标端的中间态数据同时构建两颗Merkle树并进行比较，从而实现了对于源端数据库和目标端数据库数据一致性校验。
 - 技术框架采用 SpringBoot 、 kafka中间件、debezium。
- 开源社区地址：<https://gitee.com/openGauss/openGauss-tools-datachecker-performance>

校验工具gs_datacheck – 原理（2）

- 限制与约束
 - JDK版本要求JDK11+
 - 当前版本5.1.0仅支持对源端为MySQL或openGauss，目标端也是MySQL或openGauss数据校验
 - 当前版本5.1.0仅支持数据校验，不支持表对象校验
 - MYSQL需要5.7+版本
 - 当前版本5.1.0不支持地理位置几何图形数据校验

校验工具gs_datacheck – 原理（3）

- 全量数据校验

- 将MySQL源端和openGauss目标端数据通过分片方式抽取出来后进行规整，计算（hash），并将数据推送到kafka暂存。
- Check服务即校验服务提取kafka中的数据，并对提取到的数据进行校验。
- 输出校验结果，将校验结果输出到指定路径的文件中。

- 增量数据校验

- 通过debezium监控源端数据库的数据变更记录。

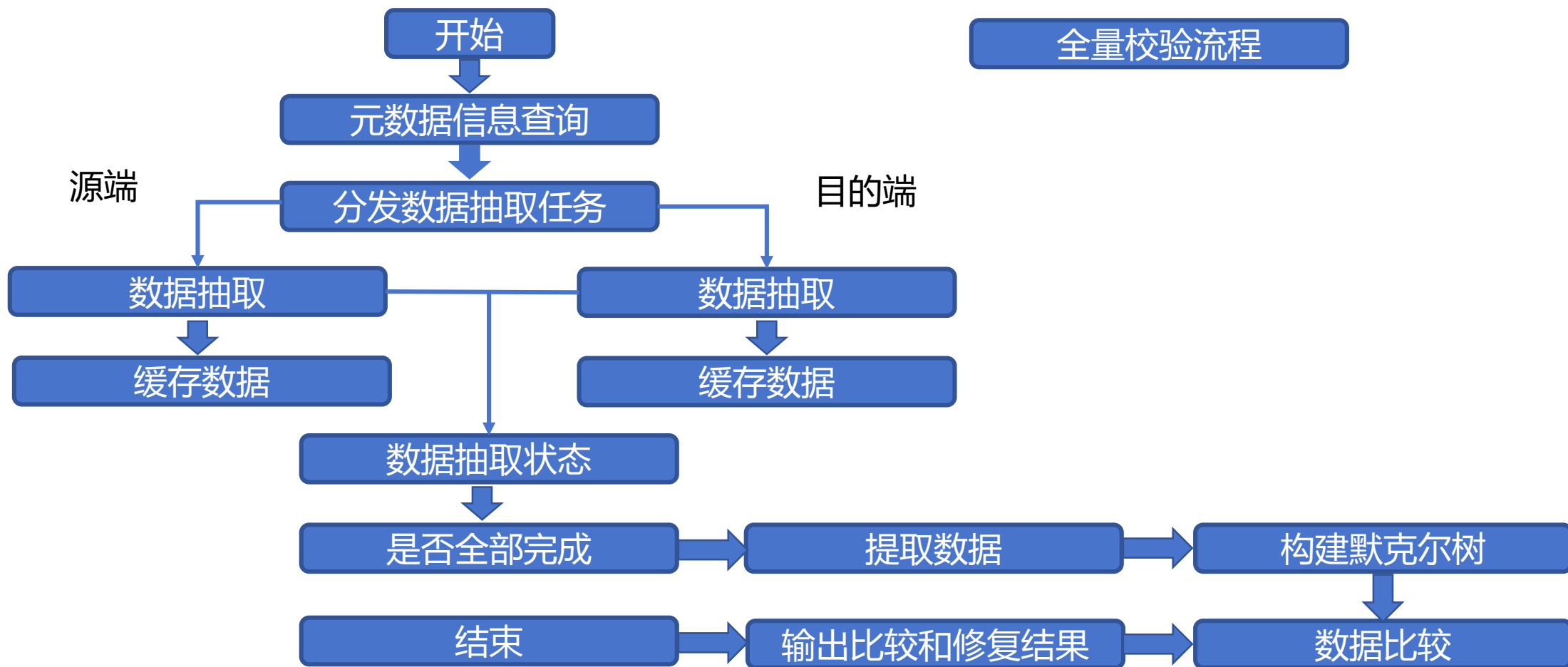
校验工具gs_datacheck – 原理（4）

- 源端抽取服务按照一定的频率（时间+数量两个纬度）处理监听到的变更记录，对变更记录进行统计合并。
- 将合并结果发送给数据Check服务。
- Check服务会对上次校验结果与当前增量数据进行合并。
- 根据合并之后的结果，Check服务发起增量数据抽取、校验、并将校验结果输出到指定路径文件。
- 规则过滤
 - 新增了表、行、列三类别的过滤规则。

全量校验 – 流程（1）

- 查询源端和目标端元数据信息`information_schema.tables`, `information_schema.columns`, 获取全部待校验表信息。
- 根据待校验表构建抽取任务和校验任务。
- 对表记录数较小的表采用全表查询方式, 对于大表则自动分解为多次抽取（通过where条件划分多个分片）。
- 将表数据抽取后进行规整（标准化处理）, 计算（hash）, 并将最终处理后的Hash数据推送到kafka暂存。表记录数较少则将topic为单一topic分区, 记录数较多则将数据存储在topic多个分区。
- 抽取服务会给每张表分别创建一个topic, 且源端和宿端分别使用不同的topic。
- Check服务以表为单位提取kafka中的数据进行校验。获取指定表源端和目标端Topic数据, 分别构建两棵默克尔树。默克尔树构建成功后, 通过对比两颗默克尔树完成表数据校验。
- 输出校验结果, 将校验结果输出到指定路径的文件中。

全量校验 - 流程 (2)



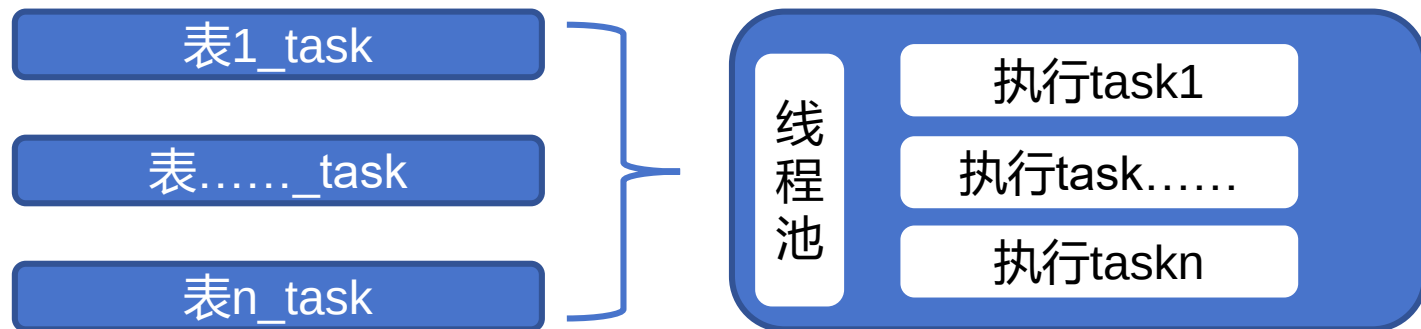
全量校验 – 数据抽取（1）

- 数据抽取服务，是根据表元数据信息构建数据抽取任务。通过JDBC方式从数据库抽取表数据，并对数据进行规整和计算。
- 并将计算结果以表为单位，存储在kafka中。每张表创建一个topic。



全量校验 – 数据抽取（2）

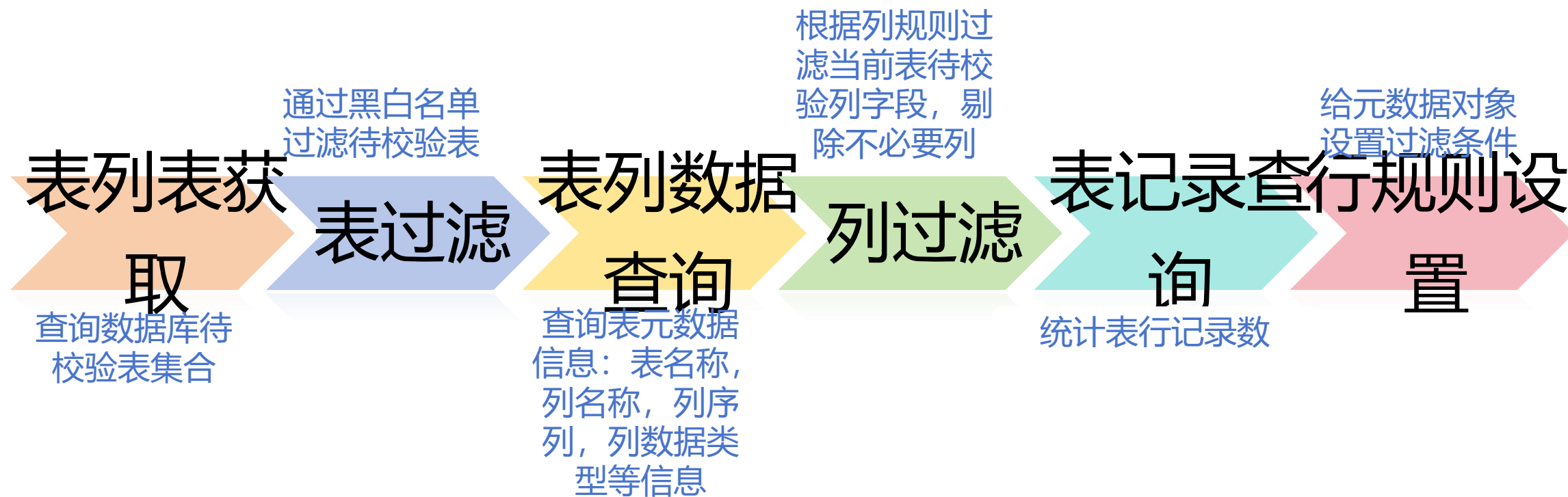
- 多表之间并行抽取，单个表根据数据量大小判断是否开启并行抽取。
- 当表记录数大于指定记录阈值时，自动对当前任务进行分片，开启并行抽取。



全量校验 – 数据校验（1）

- 数据校验HASH算法：采用xxHash算法 中性能最优XXH3算法。
- xxHash 是一种极快的哈希算法，在 RAM 速度限制下运行。它成功完成了评估哈希函数的碰撞、分散和随机性质量的SMHasher测试套件。代码是高度可移植的，并且哈希值在所有平台上都是相同的。xxHash目前有4种实现：XXH32、XXH64、XXH128、XXH3 性能最优的是XXH3。
- 过滤规则分为三类，表过滤规则，行过滤规则，以及列过滤规则。
 - 表规则，即添加表的黑白名单，通过黑白名单（正则表达式）过滤是否校验那些表。
 - 行规则，即对指定范围的表添加行记录筛选，获取表的部分数据进行数据校验。
 - 列规则，即对指定表添加列字段过滤规则，只校验当前表的部分字段数据。
 - 表和行过滤规则是根据正则表达式进行匹配，列过滤规则是根据表名进行匹配，过滤规则在抽取服务加载元数据信息时进行触发，并执行。

全量校验 – 数据校验 (2)



全量校验 – 默克尔树 (1)

- Merkle Tree是一种树，大多数是二叉树，也可以是多叉树。Merkle tree的叶子节点的Value是数据集合的单元数据或者单元数据的HASH。非叶子节点的Value是根据它下面的叶子节点值，按照HASH算法计算得出的。创建Merkle tree是 $O(n)$ 的复杂度，树高是 $\log(n)+1$ 。两个节点间数据进行比对时，从Merkle tree的根节点进行对比，根节点一致，不再做任何处理；不一样，遍历Merkle tree定位不一致的节点。定位速度快，定位的时间复杂度是 $O(\log(n))$ 。业界Cassandra采用固定的15层，即最大叶子节点数量的 2^{15} (32768) 个。实际构建过程是根据表记录规模决定最终树高，但树深度不超过15层。

全量校验- 默克尔树节点比较（2）

- 我们通过对两颗默克尔树进行比较，获取到最终差异叶子节点。对该叶子节点进行二次定位解析，获取最终的差异记录，并返回相应的行记录标识。

新增数据				删除数据				修改数据			
集合A		集合B		集合A		集合B		集合A		集合B	
1	HashA	1	HashA	1	HashA	1	HashA	1	HashA	1	HashA
2	HashB	3	HashC	3	HashC	2	HashB	3	HashC	3	HashB
3	HashC	4	HashD	4	HashD	3	HashC	4	HashD	4	HashD
4	HashD	5	HashE	5	HashE	4	HashD	5	HashE	5	HashE
5	HashE	6	HashF	6	HashF	5	HashE	6	HashF	6	HashF
6	HashF	7	HashG	7	HashG	6	HashF	7	HashG	7	HashG
7	HashG					7	HashG				

全量校验 - 默克尔树节点构建

- 由于表记录数量不确定性（表可能100条记录，或者100万条记录）树最高不能超过15层，叶子节点不能超过32768个，那么100万记录如何校验？这时候有了分桶概念，分桶就是将100万数据按照Hash规则将数据分散到32768个叶子节点中，那么每个节点会有个存储容器，我们定义为桶。分散数据到各个桶的过程，我们定义为分桶。
- 分桶是由默克尔树高度限制所决定，默克尔树高度在不超过15层时（当树高为15时，满二叉树叶子节点32768个），性能会比较好，当树高度超过15层后会导致树构建以及遍历性能下降。
- 数据分桶，将拉取的数据根据数据主键Hash值进行模运算，将数据分别添加到不同的桶中。

增量校验 – 流程

- 通过debezium监控源端数据库的数据变更记录（debezium会将源端的数据变更记录在指定Topic中）。
- 源端抽取服务按照一定的频率（时间+数量两个纬度）处理监听到的变更记录（即指定Topic），对变更记录进行统计合并。
- 以表为单位统计当前表那些记录发生变化（主键值）。
- 将合并后的变更结果发送给Check服务。
- Check服务会对上次校验结果与当前增量数据再次进行合并。
- 根据合并之后的结果，Check服务发起增量数据抽取、校验、并将校验结果输出到指定路径文件。

应用适配

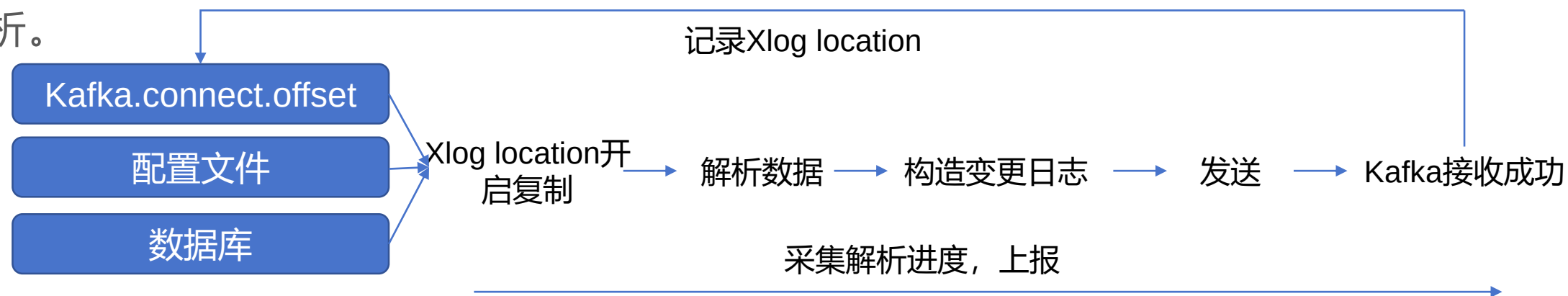
- 基于迁移评估结果，对于不兼容的SQL，应用侧进行适配改造。
- 适配改造关注点：
 - 数据类型的精度、最大长度是否一致；
 - 业务中包含的内置函数，目标库是否存在，功能是否一致；
 - 是否包含数据库方言SQL；
 - 关键字和保留字是否一致，标识符，如表名、列名、索引名、视图名、存储过程名是否包含关键字及保留字；
 - 大小写敏感是否一致，表名、列名、索引名是否区分大小写，索引名是否可重复。
- 应用改造后，需增加手动或自动化用例，进行充分自验，保证功能正常可用。

反向增量迁移gs_replicate工具介绍

- 反向增量迁移工具介绍
- Debezium反向增量迁移工具支持将openGauss数据库的增量数据实时同步至MySQL数据库。
- Debezium反向增量迁移工具由两部分组成，日志解析（source）和数据回放（sink），两部分通过kafka实现解耦。日志解析部分对openGauss的Xlog进行解析，获得Avro格式的变更记录，数据回放部分根据变更记录构造目标库兼容的sql语句，并通过jdbc接口执行。

反向增量迁移gs_replicate工具 - Source端实现原理

- 反向增量迁移source端在源库建立逻辑复制槽和发布订阅，获得二进制流式数据，进行解析。



- 日志解析串行执行，主要负责解析源库的各种数据类型，与数据库元数据一起封装为变更记录，并存到kafka中。source端支持从自定义的Xlog位置开始复制，在一个迁移任务中，正向增量迁移结束时会输出Xlog位置，反向增量迁移从该位置开始复制，保证了迁移任务由正向同步切换至反向同步的过程中数据不重不漏。

反向增量迁移gs_replicate工具 – Sink端实现原理

- 反向增量迁移sink端从kafka抽取变更记录，按表并行构造sql语句并连接到目标库执行。
- Sink端从kafka抽取变更记录时，同时监控内存中待回放数据量的大小，若数据量过大，则暂停抽取，直至待处理数据量减少到一定程度。分发数据时，不同表的变更记录优先在不同的线程中执行，若表之间有依赖，则在同一个线程执行。对于执行失败的sql语句，工具会定时输出到本地文件。

反向增量迁移gs_replicate工具使用

- 创建配置文件 —— openGauss-source.properties, openGauss-sink.properties
- 位置: confluent-5.5.1/etc/kafka/
- <https://gitee.com/openGauss/debezium/blob/master/debezium-connector-openGauss/patch/openGauss-source.properties>
- <https://gitee.com/openGauss/debezium/blob/master/debezium-connector-openGauss/patch/openGauss-sink.properties>
- 反向增量迁移工具所需依赖与正向增量迁移相同。
- 启动方法与正向增量迁移相同。

反向增量迁移gs_replicate工具约束

- openGauss 需开启逻辑复制功能，修改 GUC 参数 wal_level=logical，且仅限拥有 REPLICATION 权限的用户进行操作；
- 需要调整数据库节点下的 pg_hba.conf 以允许复制（这里的值取决于实际的网络配置及用于连接的用户）；

host	replication repuser	0.0.0.0/0	sha256
------	---------------------	-----------	--------

- openGauss 的库与逻辑复制槽一一对应，当待迁移的库发生变化时，需重新配置逻辑复制槽的名字；
- 反向增量迁移暂不支持对 DDL 操作的迁移；
- 支持的源端库需 openGauss 3.0.0 及以上版本。

反向全量数据迁移 – 数据采集模块及数据回放模块原理 (1)

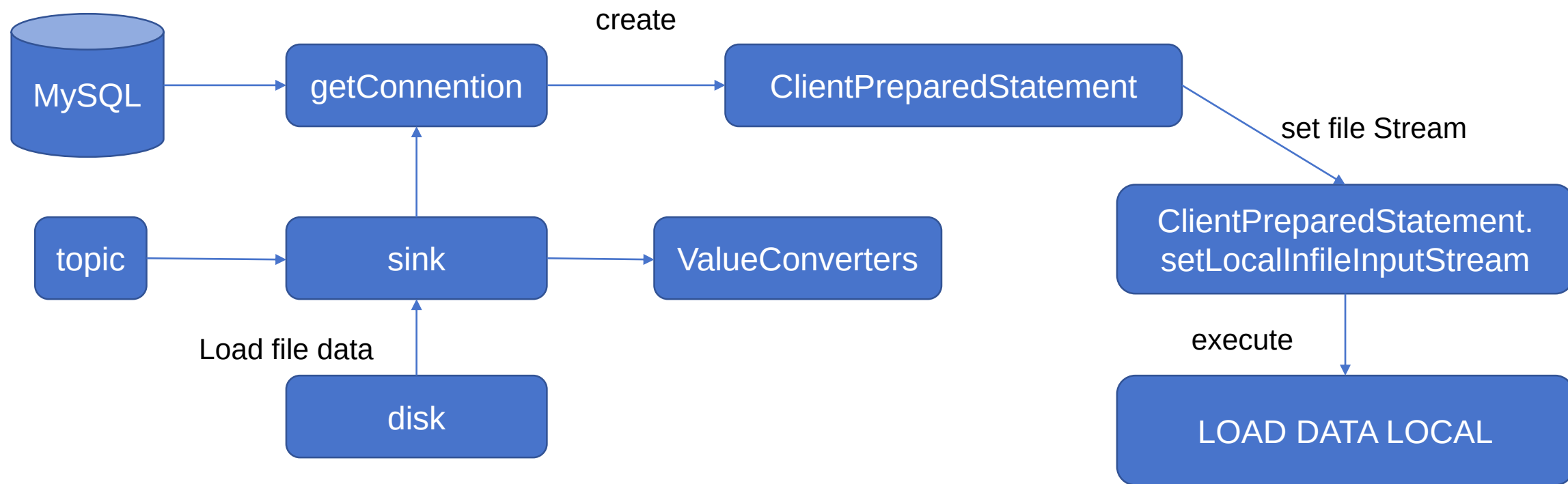
- 数据采集模块实现原理

- 通过获取库表元数据，估算划分落入文件的数据量，在采集的数据量达到估算的数据量时进行一次IO，数据落盘后将开始组装全量的kafka消息，将文件信息发送至kafka用于和回放数据模块进行交互。

- 数据回放模块实现原理

- Sink端消费kafka消息将文件数据做数据类型转换后，使用JDBC实现对数据的回放。

反向全量数据迁移 – 数据采集模块及数据回放模块原理（2）



反向全量数据迁移 – 对接反向增量迁移

- 对接反向增量迁移：增量迁移是解析xlog日志来捕获数据变更，在全量和增量对接中需要对接捕获变更的起始点，在全量迁移的过程中，有如图的几个步骤：

- 确认迁移的表
- 锁表
- 获取全局的LSN（增量捕获变更的起始点）
- 采集数据获取数据集
- 释放表锁
- 处理采集的数据集



反向全量数据迁移 – 使用及约束

- 与反向增量迁移使用方式一致，创建source和sink端配置文件
- openGauss-source.properties和openGauss-sink.properties
- Source端开关：snapshot.mode=always
- Sink端完全适配
- 约束
 - 仅支持从openGauss数据库复制全量数据迁移至MySQL数据库
 - MySQL参数设置： 需开启 local_infile = 1;
 - 继承反向增量其他的约束

数据迁移

- **组网部署**：基于迁移评估中确认的硬件资源、部署方式、运行参数等完成数据库组网部署；
- **预检查**：检查数据库的读写权限、数据库的网络连接、数据迁移源端和目标端前置参数设置等；
- **全量数据迁移**：全量对象（表、数据、索引、视图、函数、存储过程、触发器）等迁移至目标数据库；
- **全量数据校验**：对表数据及对象定义进行校验，输出校验报告，并针对不一致的对象给出修复建议；
- **增量数据迁移**：对迁移过程中的增量数据（DDL和DML）进行迁移；
- **增量数据校验**：对增量数据（DDL和DML）进行周期性校验，并针对不一致的记录给出修复建议。

试运行

- 迁移流量到目标库，对目标库进行实际业务测试，修复故障、性能调优。
- 试运行前检查：
 - 同步延迟检查、目标端增量日志检查；
 - 确认增量数据追平；
 - 停止正向增量数据迁移；
 - 删除数据库临时对象。
- 流量镜像：Attach到业务进程，镜像业务流量到目标库；
- 故障修复：监控目标库运行，修复故障；
- 性能调优：监控目标库运行，性能调优，慢SQL优化。

一键式迁移工具gs_rep_portal介绍

- 一键式迁移工具gs_rep_portal集成了全量迁移、增量迁移、反向迁移、数据校验的工具。gs_rep_portal支持一键式安装上述工具，并设定迁移任务。gs_rep_portal可以根据用户设定的迁移任务的执行计划顺序去调用相应工具完成每个迁移步骤，并能展示每个步骤的状态、进度、异常原因等。
- Portal中默认含有三种执行计划：

计划名称	包括指令
plan1	全量迁移→全量校验
plan2	全量迁移→全量校验→增量迁移→增量校验
plan3	全量迁移→全量校验→增量迁移→增量校验→反向迁移

一键式迁移工具gs_rep_portal启动

- 启动
- portal的启动命令分为几个部分：

```
sh gs_rep_portal.sh 命令 id
```

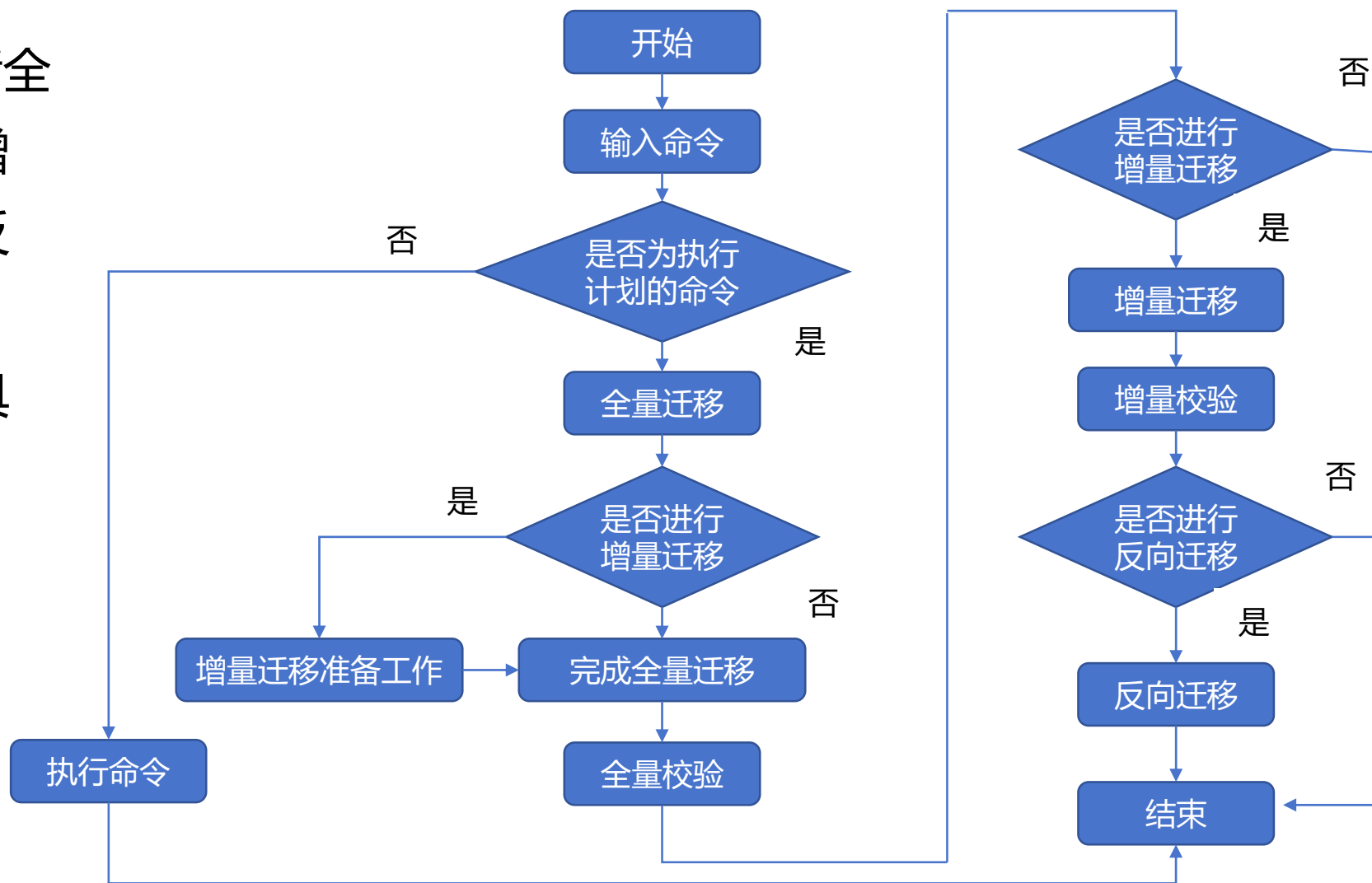
- 命令部分即为使用时的命令，包括启动kafka、启动全量迁移、启动迁移计划等。
- 每次运行portal时都需要为命令指定一个id，命令的id可以包含小写字母和数字。
- 然后这个命令执行的日志会存储在portal/logs文件夹下，名字为portal_id.log。
- 如果是迁移的命令，那么在workspace/id/logs文件夹下还有各迁移工具运行时的日志。

```
sh gs_rep_portal.sh start_mysql_full_migration k1
```

- 举例：
- 上面这个命令即为以workspace.id=k1启动mysql全量迁移。

一键式迁移工具gs_rep_portal工作流程

- MySQL全流程迁移包括全量迁移、全量校验、增量迁移、增量校验、反向增量迁移五个阶段，可根据执行计划决定具体的迁移步骤。



一键式迁移工具gs_rep_portal注意事项

- 如果执行的计划里面同时包含全量迁移和增量迁移，portal需要从全量迁移中获取快照点并提前启动增量迁移的source端，这样可以提高增量迁移的性能，节省等待时间，所以在进行全量迁移的时候会有增量迁移的准备工作。
- 增量迁移是持续性过程，如果用户不输入停止命令，那么增量迁移过程将一直进行。
- 这里的反向迁移特指反向增量迁移，也是持续性过程。用户在使用过程中需要另开一个命令行窗口去执行停止增量迁移、启动反向迁移、停止计划等相关命令。
- 除了单独执行全量迁移外，其他迁移工具都需要依赖kafka，portal在安装迁移工具时自动运行了start_kafka命令，所以直接启动迁移计划即可，如果kafka出现问题，用户可以使用stop_kafka命令停止kafka，删除缓存文件后使用start_kafka命令重新启动。

一键式迁移工具gs_rep_portal – 示例（1）

- 如何执行完整的迁移计划（以centos7系统为例）
- 在指定链接下载portal安装包

```
wget -c https://openGauss.obs.cn-south-1.myhuaweicloud.com/latest/tools/centos7/PortalControl-5.1.0-x86\_64.tar.gz
```

- 解压gs_rep_portal安装包

```
tar -zxvf PortalControl-5.1.0-x86_64.tar.gz
```

- 在/ops/portal/config目录的toolspath.properties文件中修改安装路径，然后使用安装命令

```
sh gs_rep_portal.sh install_mysql_all_migration_tools 1 &
```

- 在/ops/portal/config目录的migrationconfig.properties文件中修改迁移参数

一键式迁移工具gs_rep_portal – 示例（2）

- 如何执行完整的迁移计划（以centos7系统为例）
- 指定新的workspace.id为2启动迁移计划3

```
sh gs_rep_portal.sh start_plan3 2 &
```

- 程序将自动运行至增量迁移和增量校验同时开启中，另开窗口让workspace.id为2的任务停止增量迁移，此时程序进入等待状态，之后可以启动反向迁移或直接停止计划

```
sh gs_rep_portal.sh stop_incremental_migration 2 &
```

- 启动反向迁移，此时程序进入反向迁移状态，之后可以停止计划

```
sh gs_rep_portal.sh run_reverse_migration 2 &
```

- 停止workspace.id为2的计划

```
sh gs_rep_portal.sh stop_plan 2 &
```


工具一体化平台DataKit简介

- DataKit定位为openGauss的数据全生命周期生产力工具，支持数据全生命管理，覆盖openGauss安装部署、数据开发、智能运维、数据迁移等阶段Debezium反向增量迁移工具支持将openGauss数据库的增量数据实时同步至MySQL数据库。
- DataKit是openGauss的一个工具集，采用B/S（即Browser/Server, 浏览器/服务器）软件架构，提供基础的服务器、数据库实例等软硬件资源管理能力；同时支持通过按需部署功能插件扩展能力，目前社区提供安装部署、数据迁移、数据开发、智能运维等功能插件。
- DataKit也是一个工具开发平台，支持用户根据插件接口和规范实现自己的特有功能。功能插件支持动态加载到DataKit上运行，也支持从DataKit上动态卸载。插件支持调用平台和其他插件的开放接口获取相应数据和功能，以实现快速构建特色功能。

工具一体化平台DataKit主要功能

- 主要功能
- 插件平台
- 用于插件开发的前后端一体的开发框架
- MySQL数据迁移工具插件实现
- 安全中心（账号管理、角色权限管理和白名单管理）
- 插件管理模块
- 日志中心，包括操作日志和系统日志（日志级别/切割规则/保留天数的设置和日志下载）
- 国际化（中/英）
- 主题切换（黑/白）
- 基础模块（用户中心、修改个人密码、登录/登出）

工具一体化平台DataKit - 数据开发

- DataKit-安装部署

- 一键部署openGauss，让用户快速安装和使用openGauss
- 自定义安装openGauss，支持单机、主备、分布式的部署形式，支持极简版、轻量版、企业版不同版本的安装。
- 支持数据库的升级和备份功能

- DataKit - 数据开发

- 业务开发是一个Web化的通用集成开发环境（Web Data Studio），它以插件形式运行于DataKit中，支持对openGauss 数据库进行管理。
- 业务开发（Web Data Studio）以图形化界面形式提供数据库的对象管理、数据操作、函数/存储过程开发等能力，方便数据库开发和维护人员管理数据库。
- 业务开发（Web Data Studio）还支持函数和存储过程调试，数据库开发人员可以通过断点操作查看每一步的执行过程、变量取值，快速精确定位到函数和存储过程的异常所在点，降低排错难度。

工具一体化平台DataKit – 智能运维

- 智能运维主要提供openGauss运维相关的功能，具体由三个功能插件组成：
 - 实例监控：支持对openGauss实例及其运行环境的资源消耗情况进行监控以及问题分析，主要功能包括：实时采集和展示数据库本身以及所在系统的运行指标、指标的上钻下探关联分析、下钻Top SQL查看SQL详情、生成WDR报告、OS和数据库配置参数查看及优化建议等。
 - 日志检索：支持采集和展示多个openGauss实例和OS日志，通过一个统一的界面对这些日志进行按照类型、级别、实例、关键字、时间等条件进行集中检索，并支持日志上下文关联查询，方便从日志中快速发现数据库异常情况。
 - SQL诊断：支持对SQL运行期间进行全栈跟踪寻找SQL发生异常的根因，支持对SQL执行计划、数据库系统参数等方面进行分析，并最终给出智能的诊断优化建议。

工具一体化平台DataKit – 数据迁移（1）

- 数据迁移插件提供了MySQL迁移所需的所有能力，包括：
 - 对象迁移：支持将MySQL的数据库对象，包括：表、索引、约束、视图、存储过程、函数、触发器等，迁移到openGauss。
 - 全量数据迁移：支持将MySQL全量数据并发迁移到openGauss，迁移性能达到300MB/s。
 - 增量数据迁移：通过读取MySQL侧的BinLog并转化成SQL语句在openGauss侧回放，支持并行回放，保证事务一致性，迁移性能达到30000TPS。
 - 全量数据校验：针对源DB和目标DB的全量数据，以表为单位进行校验，找出差异记录，给出修复建议，输出校验报告。

工具一体化平台DataKit – 数据迁移（2）

- 数据迁移插件提供了MySQL迁移所需的所有能力，包括：
 - 迁移任务管理：用户可以通过配置迁移任务，将单个或多个源端数据源中的数据迁移至目标数据源。
 - 迁移过程监控：用户可以实时查看迁移进度、日志和错误信息，并进行相应的处理。
 - 并发执行：用户可以创建多个迁移任务并行执行，以提高迁移效率。
 - 资源动态分配：系统能够根据最大化利用原则，合理分配运行机器，以实现多任务并行迁移。
 - 迁移模式包括离线模式和在线模式，其中离线模式包括全量迁移、全量校验两个阶段，在线模式包括全量迁移、全量校验、增量迁移、增量校验、反向迁移五个阶段。

工具一体化平台DataKit – 使用步骤（1）

- 仓库地址：<https://gitee.com/openGauss/openGauss-workbench>
- 下载地址：
- <https://openGauss.obs.cn-south-1.myhuaweicloud.com/latest/tools/Datakit/Datakit-5.0.0.tar.gz>
- 使用步骤：
 - 创建config/application-cus.yml文件

```
keytool -genkey -noprompt -dname "CN=openGauss, OU=openGauss, O=openGauss, L=Beijing, S=Beijing, C=CN" -alias openGauss -storetype PKCS12 -keyalg RSA -keysize 2048 -keystore /ops/ssl/keystore.p12 -validity 3650 -storepass 123456
```

工具一体化平台DataKit – 使用步骤（2）

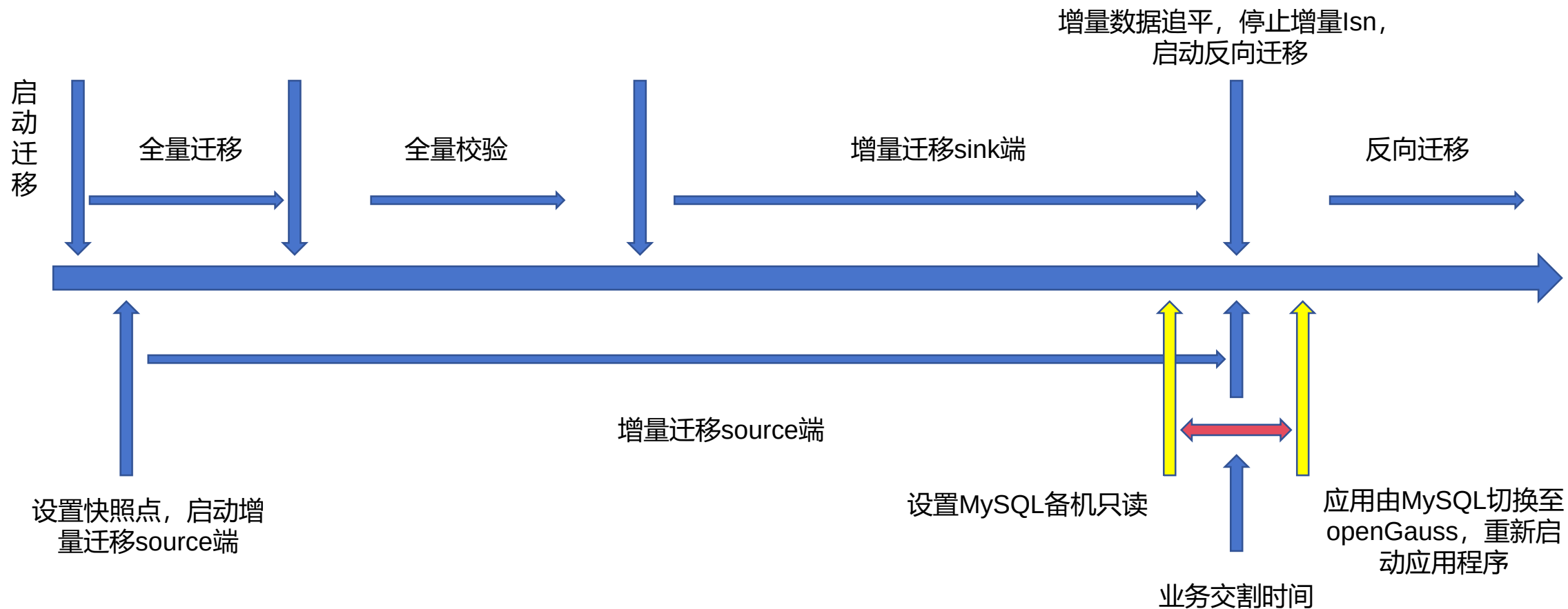
- 生成密钥

```
nohup java -Xms2048m -Xmx4096m -jar visualtool-main.jar --spring.profiles.active=cus > ./visualtool-main.out 2>&1
```

- 启动主程序
- 在界面上安装插件

工具一体化平台DataKit – 工作流程

- MySQL全流程迁移包括全量迁移、全量校验、增量迁移、增量校验、反向增量迁移五个阶段，可通过DataKit平台完成一键式迁移。



生产割接

- 启动反向迁移，业务部署在目标数据库，启动应用程序；
- 反向迁移作为一种数据逃生的通道，可保证源端和目标端数据一致，防止目标端数据库出现问题时，可及时将应用切回至源端数据库，保证数据正常回退，避免对业务造成影响；
- 利用监控工具对数据库各项指标进行监控，及时发现业务运行的异常点；
- 当业务在目标数据库平稳运行一段时间后，可将源库下线，业务完全应用于目标数据库。